

APARAJIT: An Edge-compute System-on-chip

Madhav Desai
Department of Electrical Engineering
IIT Bombay

July 17, 2026

Overview

- ▶ APARA, the edge compute accelerator.
 - ▶ instruction set architecture.
 - ▶ data and instruction memory.
 - ▶ DMA engine.
- ▶ APARAJIT a minimal system on chip:
 - ▶ Control processor (AJIT processor).
 - ▶ Accelerator (APARA accelerator).
 - ▶ DMA controller.
 - ▶ Switch.
 - ▶ Memory.
- ▶ Programming APARAJIT.
- ▶ Demonstration of FPGA prototype and early performance data.
- ▶ Road-map.

APARA architecture

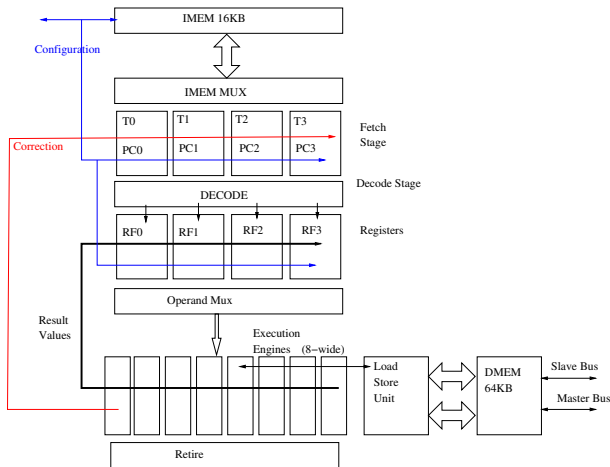


Figure: APARA structure

APARA features

- ▶ A memory-coupled accelerator.
- ▶ Disjoint instruction and data memories.
- ▶ Four execution threads multiplexed onto a single hardware pipeline.
- ▶ Hardware pipeline: Bundles of instructions with up to 8 instructions per bundle.
- ▶ Supported types: i4/i8/i16/i32/i64, f4/f8/f16/f32/f64
- ▶ Per-thread 32x 64-bit registers, Per-thread program counters, Per-thread return address stack.
- ▶ Instructions:
 - ▶ Memory load/store: from byte to 32-bytes.
 - ▶ Scalar and vector instructions, including dot-product.
 - ▶ Branching and subroutines.

Multi-threaded APARA: functional description

```
// 4 threads, 4x PC, 4x REGSETS, 4x RAS, 4x ERR
PC[1 to 4] = initvalues , REGS[1 to 4],
RAS[1 to 4] = empty, ERR[1 to 4] = 0, ACTIVE=1
while(1)
{
    bundle := fetch_bundle_from_imem(PC[ACTIVE])
    ops      :=
get_ops_needed_by_bundle(bundle, REGS[ACTIVE])
    out_values :=
        execute_instructions_in_bundle(bundle, ops)
    PC[ACTIVE], REGS[ACTIVE], ERR[ACTIVE] :=
        update_registers_pc_ras_error (out_values)
    ACTIVE = next_active (ACTIVE, ERR[1 to 4]);

    if(ACTIVE==0) break;
}
```

Abstract machine: mcode program

- ▶ Up to 4 threads are active in the abstract machine. They are multiplexed onto the execution pipeline.
- ▶ Each thread has its own state: program counter, register file, return address stack, error status.
- ▶ Each thread executes an Mcode program.
- ▶ A Mcode program is a sequence of instruction bundles.
- ▶ Each bundle can have 1/2/4/8 instructions.
- ▶ Instructions in a bundle are executed in parallel using a synchronous fetch-execute-write sequence which guarantees no race conditions.
- ▶ A single bundle can have at most one control transfer instruction.
- ▶ Two instructions in a bundle cannot write to the same register.

Bundle

```
label:  
  ||  
    <instr 1>  
    <instr 2>  
    <instr 3>  
    ...  
    <instr 8>  
  ;
```

Can have 1/2/4/8 instructions in each bundle.

Instruction set: instructions in a bundle

- ▶ Each instruction is 32-bits wide.
- ▶ Instruction classes:
 - ▶ ALU instructions.
 - ▶ Integer: $+$, $-$, $*$, $/$, *or*, *and*, *xor*, *nor*, *nand*, *xnor*, $<<$, $>>$.
 - ▶ Float: $+$, $-$, $*$, $/$, square-root.
 - ▶ Load/Store instructions.
 - ▶ Control transfer instructions: branch, call, return, conditional move.
 - ▶ Vector instructions: vector add, subtract, multiply, scale, reduce, dot-product.
 - ▶ Miscellaneous instructions: set, slice, pack, null, nop, cast.
- ▶ Control transfer instructions: branch, call, return.

Types

Integer (signed, unsigned, vector)

signed: (\$i4) (\$i8) (\$i16) (\$i32) (\$i64)

unsigned: (\$u4) (\$u8) (\$u16) (\$u32) (\$u64)
(\$u128) (\$u256)

float

(\$f4) (\$f8) (\$f16) (\$f32) (\$f64)

vector: (\$vi4) (\$vi8) (\$vi16) (\$vi32)

(\$vu4) (\$vu8) (\$vu16) (\$vu32)

(\$vf4) (\$vf8) (\$vf16) (\$vf32)

Note: 128 and 256 bit types are used only during data transfer with memory, not for arithmetic.

Orthogonalization: op-code and type

+ \$r4 (\$i16) \$r1 \$r2

+ \$r4 (\$i32) \$r1 \$r2

+ \$r4 (\$i64) \$r1 256

+ \$r4 (\$u16) \$r1 \$r2

+ \$r4 (\$u32) \$r1 \$r2

Vector ALU instructions

`$v * $r8 ($vi32) $r8 $r12`

Same as ALU instructions except that

- ▶ immediate operands not possible.
- ▶ source and destination types are vectors.
- ▶ Replicate flag is provided instead of immediates.

Vector ALU DOT instruction

```
$dot                $r24 ($vi16) $r8  $r12  
$dot $accumulate    $r24 ($vi16) $r8  $r12
```

A dot-product on the two source vectors which produces a destination value of the specified type ($rd = rs1.rs2$). The accumulate version does $rd = rs1.rs2 + rd$.

Vector VREDUCE instruction

```
$vreduce + $r8 ($vi32) $r12
```

The register \$rs1 is a vector. The elements of the source vector are added using precision specified by dest-type, and the result is saved into \$rd.

Casts: the CAST instruction

```
$cast ($f32)  $r8 ($f16) $r7
```

```
$cast ($vf32) $r8 ($vf16) $r7
```

```
$cast ($vf8)  $r8 ($vf16) $r9
```

The bits in *rs* are mapped to bits in *rd* via a cast operation. For scalars, it is the normal cast (as in the first example). For vectors, as in the second example, the vector in *\$r7* is mapped to *\$r8* and *\$r9*. Two output registers are used. In the third example, the destination vector is filled with 8 values and this needs us to read a multiplicity of two registers *\$r9* and *\$r10*.

The multiplicity of in or out registers should be at most 2.

Casts: the PACK instruction

```
$pack $r8 64 32 $r4
```

```
$pack $r8 32 16 $r4
```

In the first example, The 32 bit values in \$r4, \$r5 are concatenated to produce a 64-bit result value in \$r8. The concatenated values must fit into a single destination register (in this case \$r8). The second example is similar but two 16-bit values are packed into a 32-bit value.

Data transfer: the LOAD instruction

```
$ld ($i64)  $r3 [$r1 + $r0]  
$ld ($u256) $r4 [$r1 + $r2 ++1]  
$ld ($u64)  $r8 [$r1 + 256]
```

The specified number of bits are fetched from memory and kept in registers starting at \$rd. Up to 64 bits use one destination register. Higher values of number of bits (max value is 256) use two to four destination registers. Auto-increment, if specified increments rs2 by the specified value times the size of transfer in bytes.

Data transfer: the STORE instruction

```
$st <dest-type> [ <rs1> + <rs2> <auto-incr>?] <rd> 0R  
$st <dest-type> [ <rs1> + 16] <rd>
```

```
$st ($i32) [$r5 + $r6] $r8  
$st ($i128) [$r5 + $r6] $r8
```

The specified number (in dest-type) of bits are moved from the specified rd register into memory. Note that multiple source registers may be used (e.g. for 128 bits we need r8,r9 to be written to memory). Register indices and memory addresses must be aligned values.

Conditional move: CMOV

```
$cmov ($uint32) $r5 > --1 $r6 $r7
```

In the example, if \$r5 is greater than 0, then \$r7 is moved to \$r6.
The \$r5 register value is decremented.

Control Transfer: the BRANCH instruction

```
? ($i32) $r5 > --1 $goto loop_begin
```

The example instruction above can be interpreted as follows:
Register \$r5 is compared with 0 and if it is > 0 (as a 32-bit signed integer) then control is transferred to the label `loop_begin`. The value of \$r5 is decremented by 1 after the comparison. The auto-decrement (or increment) is optional.

Control Transfer: the CALL instruction

```
$call foo
```

```
$call $r5
```

The expected next PC is saved on the return address stack and control is passed to the bundle with label foo or with the address ($PC + \$r5$) (here $\$r5$ is interpreted as a 32-bit signed integer).

Control Transfer: the RETURN instruction

`$return`

On a return instruction, the return address stack is popped and control is passed to the popped address.

Miscellaneous: the SET instruction

```
$set $r6 2 _hffff
```

The 16-bit init-value *0xffff* is inserted into r6 at bits [31:16]. SET is meant to be used for initialization, 16 bits at a time.

Miscellaneous: the NULL, NOP and HALT instructions

`$null`

`$nop`

`$halt`

The NULL and NOP instructions do nothing. The HALT instruction halts the machine (after executing all instructions in the bundle in which the HALT occurs).

Tools

- ▶ `mcode_align`: insert null instructions to align bundles to required boundaries.
- ▶ `mcode_assemble`: generate hex instructions for instruction memory.
- ▶ `mcode_disassemble`: from hex back to assembly.
- ▶ `mcode_run`: an emulator.

Code example: simple dot product

SEE THE CODE.

APARAJIT 1P1E: first prototype

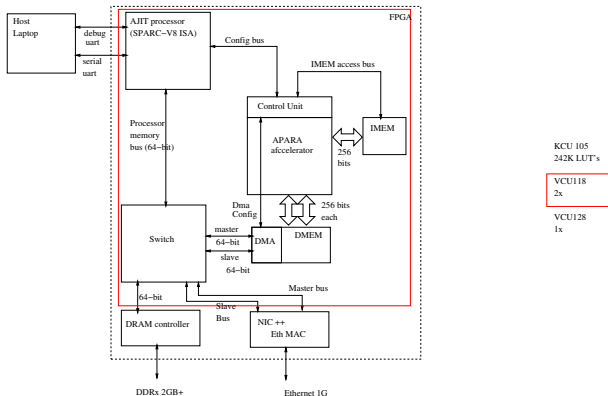
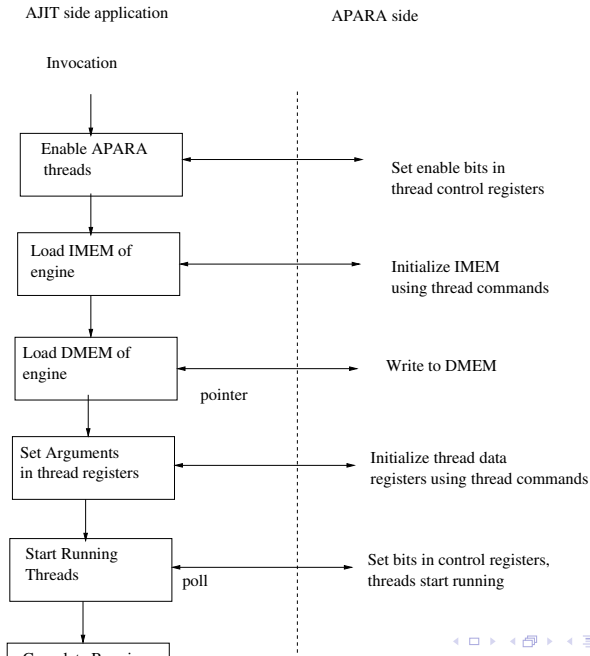


Figure: APARAJIT 1P1E prototype (with Ethernet and PCIe interfaces shown)

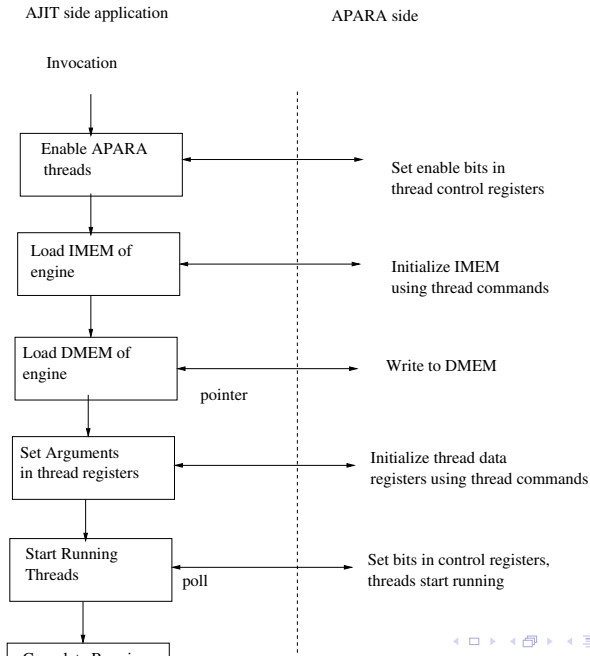
APARAJIT 1P1E Prototype: Notes

- ▶ Using the configuration interface, we can start up to four execution threads by supplying an initial PC to each of them.
- ▶ Using the configuration interface, we can start up to four DMA threads by supplying source and destination parameters to the DMA thread.
- ▶ LUT count for processor (with FPU, MMU, L1 caches): 80K LUT.
- ▶ LUT count for accelerator engine: 35K LUT per lane.
 - ▶ Each lane: 2x FP32 mul, add, FP64 mul, add, 64-bit integer add, mul, 4/8/16 bit integer and FP dot product unit.

APARAJIT 1P1E Prototype: Simple Application Flowchart



APARAJIT 1P1E Prototype: peer-to-peer queueing model



APARAJIT 1P1E Prototype: DMA slicing

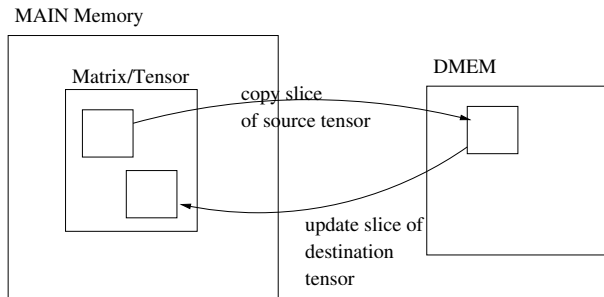


Figure: DMA Slicing

APARAJIT 1P1E Prototype: DMA slicing demo

- ▶ Configure a main memory to data memory DMA transfer on a DMA thread.
- ▶ Start the DMA thread.
- ▶ Wait until the DMA thread is completed.
- ▶ Configure a data memory to main memory DMA thread.
- ▶ Start the DMA thread.
- ▶ Wait until the DMA thread is completed.
- ▶ Repeat for each thread.

APARAJIT 1P1E Prototype: DMA performance demo

- ▶ Configure a main memory to data memory DMA transfer on a DMA thread.
- ▶ Start the DMA thread.
- ▶ Wait until the DMA thread is completed.
- ▶ Configure a data memory to main memory DMA thread.
- ▶ Start the DMA thread.
- ▶ Wait until the DMA thread is completed.
- ▶ Repeat for each thread.
- ▶ Observed performance: 1.2K cycles to transfer 8KB from main memory to local memory.
- ▶ Observed performance: 1.1K cycles to transfer 8KB from local memory to main memory.

APARAJIT 1P1E Prototype: 32-bit integer DOT-product 2K x 2K demo

- ▶ Set up initial conditions of registers (source pointers of two arrays, number of elements).
- ▶ Start four exec threads: all do the same work.
- ▶ Wait until all four threads have completed.
- ▶ Dump registers.
- ▶ Measure timings (compare with processor: 31X speedup).

APARAJIT 1P1E Prototype: 8-bit DOT-product 8K x 8K demo

- ▶ Set up initial conditions of registers (source pointers of two arrays, number of elements).
- ▶ Start four exec threads: all do the same work.
- ▶ Wait until all four threads have completed.
- ▶ Dump registers.
- ▶ Measure timings (compare with processor: 155X speedup).

APARAJIT 1P1E Prototype: queueing network based application architecture

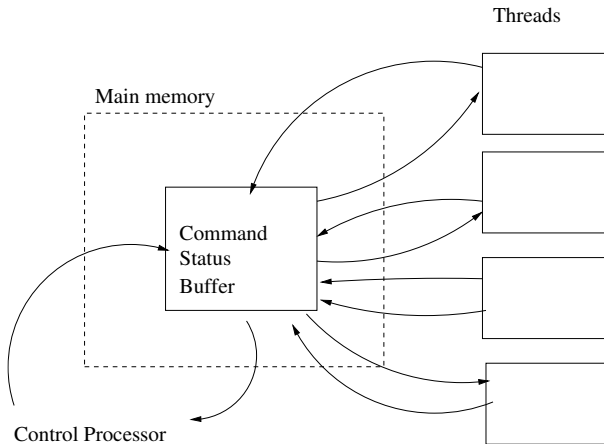


Figure: Queueing

Beyond APARAJIT 1P1E Prototype: 2P2E, 4P2E, 4P4E etc.

- ▶ Add Ethernet, PCIE interfaces to SOC.
- ▶ mPnE: m processors, n engines.
 - ▶ 2P1E, 2P2E, 4P1E, 4P2E etc...
- ▶ Variants: with 1/2/4 lanes per accelerator for deep embedded applications.
- ▶ Clusters to scale beyond 4P4E.

Software for APARAJIT SOC

- ▶ Bare metal drivers.
- ▶ RTOS port to SOC: micro-kernel based OS identified (Redkill, NuttX).
- ▶ Software library: BLAS, FFT, AIML operations.
 - ▶ Initially in assembly.
 - ▶ Eventually: compile from high level language.

Collaborations: APARAJIT SOC

- ▶ Accelerator kernel routines.
- ▶ Accelerator evaluation/use in systems.
 - ▶ as building block.
- ▶ Domain specific use cases: bio-medical, robotics.
- ▶ Other ideas?